

Parallel Project Report: Evaluation of Distributed Memory SpMV under Different Decomposition Strategies

Francesco Brunelli (ID: 245252)

Email: francesco.brunelli@studenti.unitn.it

Course: Introduction to Parallel Computing (2025-2026)

Abstract—Sparse Matrix Vector Multiplication (SpMV) is a computationally challenging problem due to its dependence on both matrix structure and hardware characteristics. This work evaluates the performance of three decomposition strategies: 1D cyclic, 1D block and 2D Cartesian block across varying MPI process counts. Five matrices from the SuiteSparse collection are used for strong scaling evaluation, while synthetic matrices are used for weak scaling. The results show that the highest performance improvements are not always achieved with the highest number of MPI processes, but depend on both the decomposition strategy and the matrix structure. The 1D cyclic decomposition performs better on highly irregular matrices despite poor cache locality, whereas 1D and 2D block strategies achieve higher performance on regular matrices thanks to balanced workloads and better cache utilization. These results highlight the importance of selecting an appropriate matrix partitioning strategy based on matrix characteristics in order to optimize parallel performance.

Index Terms—Sparse Matrix-Vector Multiplication (SpMV), Compressed Sparse Row (CSR), COO, Message Passing Interface (MPI)

I. INTRODUCTION

Sparse Matrix Vector Multiplication (SpMV) is a fundamental kernel in many scientific and industrial applications. However, due to its strong dependence on both the structure of the input matrix and the characteristics of the underlying computational architecture, the application of appropriate optimization and parallelization techniques is essential to avoid bottlenecks [1]. Parallel SpMV has been widely studied, with 1D block and cyclic decompositions commonly used for their simplicity, and 2D Cartesian block decompositions proposed to improve load balance and reduce communication. This work aims to compare these widely used 1D and 2D strategies under identical conditions to quantify their tradeoffs. Using the MPI library, this paper provides an overview of the performance achievable by different matrix decomposition techniques on a distributed-memory system, considering varying numbers of MPI processes and analyzing their respective strengths and limitations. Since sparse matrices, by definition, contain a small number of nonzero elements (NNZ), compression is typically required to improve the efficiency of the SpMV kernel [2]. Two of the most commonly used storage formats are the Coordinate (COO) format and the Compressed Sparse

Row (CSR) format. All matrices used in this study were either obtained from the SuiteSparse Matrix Collection [6], or randomly generated in MatrixMarket format, and thus were initially provided in COO representation. The main contributions of this work are: (i) the performance characterization of SpMV under three different matrix decomposition strategies (1D Cyclic Decomposition, 1D Block Decomposition, 2D Block Decomposition); (ii) an evaluation of their scalability across different matrices and numbers of MPI processes; and (iii) a comparison against a sequential baseline. The results illustrate how different decomposition strategies interact with matrix structure and process count, highlighting the advantages and limitations of each approach. The remainder of this paper is organized as follows: Section II introduces the COO and CSR formats, provides an overview of the MPI library, and describes the decomposition strategies adopted in this work. Section III describes the experimental methodology, including the code structure and the toolchain used for data collection. Section IV presents and discusses the obtained performance results. Finally, Section V summarizes the main conclusions.

II. BACKGROUND

This section provides the background necessary to understand the sparse matrix formats (COO and CSR), the MPI library and the matrix decomposition strategies used in this study.

A. COO Sparse Matrix Format

The matrices used in this study were initially provided in MatrixMarket files and therefore represented in the Coordinate (COO) format. In COO, each non-zero element is represented by its value along with its row and column indices, stored in separate arrays [3]. While simple, COO is less efficient in SpMV computation, so all matrices were converted to CSR format for the experiments.

B. CSR Sparse Matrix Format

The Compressed Sparse Row (CSR) format is a widely used sparse matrix representation. Like COO, it uses three arrays: one for non-zero values, one for column indices and a third one for the starting index of each row's nonzero elements [3]. This structure enables more efficient memory access during SpMV computations.

C. The MPI Library

Parallel computing systems are commonly classified as shared-memory or distributed-memory. In shared-memory systems multiple threads communicate by reading and writing data in a common address space, whereas in distributed-memory systems each process owns a private memory and communication is performed explicitly through message passing. The Message Passing Interface (MPI) is a widely used standard for programming distributed-memory systems, providing an efficient interface for communication and synchronization among processes [5].

D. Matrix Partitioning Strategies

In this study, three different matrix decomposition strategies have been considered:

- **1D Cyclic Decomposition:** matrix rows are distributed among the MPI processes in a round-robin manner, such that row i is assigned to process $i \bmod p$. This approach can help mitigate load imbalance caused by irregular distributions of NNZ elements.
- **1D Block Decomposition:** each process owns a contiguous set of rows and the nonzero elements within those rows. In the simplest case, the n rows of the matrix are evenly divided among p processes, with each process owning approximately n/p consecutive rows. This distribution is commonly used in many matrix libraries. However, while it balances the number of rows, it does not necessarily balance the number of nonzero elements across processes [4].
- **2D (Cartesian) Block Decomposition:** as shown in Algorithm 1, both the rows and columns of the matrix are partitioned into contiguous blocks following a Cartesian structure. The cartesian communicator was created using standard MPI routines such as `MPI_Dims_Create` and `MPI_Cart_Create`. Each process owns a rectangular submatrix defined by the intersection of a row block and a column block. Although this approach can reduce communication compared to 1D decompositions, the distribution of NNZ elements among processes still depends on the matrix structure and it is not guaranteed to be balanced.

III. METHODOLOGY

This section describes the methods and code used and the criteria adopted to gather the necessary data for this work.

A. Matrix parsing and COO loading

In our implementation, the root process (rank 0), is responsible for reading the input matrix from a MatrixMarket file, loading the NNZ entries in COO format, and distributing them among MPI processes according to the selected matrix decomposition strategy. MatrixMarket files encode both numerical type (real, complex, integer, pattern) and the structural properties of the matrix (general or symmetric). General matrices are loaded directly into COO format. For symmetric matrices, only the upper or lower triangular part is stored, so

Algorithm 1

procedure 2D_PARTITIONING

```
Split MPI processes into master (rank 0) and workers
Create a 2D Cartesian communicator among workers
if rank == 0 then
    Read matrix in COO format
    Compute row and column block sizes
    for each row block  $i$  and column block  $j$  do
        Extract submatrix corresponding to block  $(i, j)$ 
        Send submatrix dimensions and nonzeros
        to worker  $(i, j)$ 
    end for
    Broadcast input vector to all workers
else
    Receive local submatrix dimensions and NNZ data
    from master
    Convert local COO submatrix to CSR format
end if
    Perform local SpMV on assigned block
    Reduce local outputs to obtain the final result
end procedure
```

the missing symmetric entries are replicated during loading. Pattern matrices do not include numerical values; therefore, a default value (e.g., 1.0) is assigned to each NNZ entry during the loading process.

B. SpMV Computation and output gathering

Although COO is convenient for loading, it is inefficient for SpMV due to irregular memory access patterns. For this reason, once each process has received its assigned portion of the matrix, the local COO representation is converted to CSR, which enables contiguous row access and improves cache locality. For each experiment, a random input vector, with dimension equal to the number of matrix columns, is generated and made available to all MPI processes. Each process involved in computation then performs SpMV on its local CSR matrix using the standard row-wise kernel, storing the partial results in an output array initialized with zeros. While in the 1D Cyclic decomposition all processes, including the root, compute their local portion of the matrix; in 1D Block and 2D Block decompositions, rank 0 acts only as coordinator, distributing matrix chunks to worker processes following a master-workers logic, while the workers perform the computation. After local computation, the partial output arrays are combined using `MPI_Allreduce` with a summation operator. This operation correctly assembles the final result vector, since each processor contributes NNZ values only for the rows it owns, while other entries remain zero.

C. Experimental Automation Scripts

To streamline data collection, two categories of scripts were developed:

1) **Bash scripts**: used to automate the execution of experiments across different numbers of MPI processes, ranging from 2 to 128 in powers of two. These scripts reduce manual intervention and help prevent walltime overruns.

2) **PBS scripts**: responsible for job submission to the cluster's compute nodes, collecting execution outputs, and organizing results into structured folders for subsequent analysis

These scripts automatically collect performance metrics such as computation time, communication time, memory usage, floating point operations per second (FLOPS) and statistics related to the distribution of NNZ elements across different MPI ranks. These metrics are gathered for all combinations of processor counts and partitioning strategies considered in this study.

IV. EXPERIMENTS AND SYSTEM DESCRIPTION

A. Experimental Setup

All experiments were executed on a multi-socket x86_64 server equipped with four Intel Xeon Gold 6140M processors (18 cores per socket, 72 cores in total) and 4 NUMA nodes. The operating system is CentOS Linux 7, and mpicxx (GCC 9.1.0) was used for compilation with flags `-std=c++11 -O3`.

TABLE I: Hardware and Software Configuration

Component	Specification
Architecture	x86_64
CPU Model	Intel(R) Xeon(R) Gold 6140M CPU @ 2.30GHz
Sockets	4
Cores per socket	18
Total cores	72
Threads per core	1
NUMA nodes	4
L1 cache	32 KB (I) + 32 KB (D) per core
L2 cache	1 MB per core
L3 cache	25 MB per socket
Operating System	CentOS Linux 7 (Core)
Compiler	mpicxx (GCC 9.1.0, -std=c++11 -O3)

B. Metrics

Computation time: Maximum execution time of the partial CSR x V multiplication across all involved MPI processes, expressed in milliseconds (ms); 90 percentile of 100 runs

Communication time: Maximum communication time across all MPI processes, expressed in seconds (s); 90 percentile of 100 runs

FLOPS: Floating-point operations per second; expressed in GFLOPS; 90 percentile of 100 runs

(Strong) Scaling: Fixed size, different number of MPI processes

(Weak) Scaling: Matrix size proportional to the number of MPI processes

Speedup: Compared to sequential execution

C. Dataset

For the strong scaling experiments, we use a set of five matrices selected from the SuiteSparse Collection [6] (Table 2). For weak scaling experiments, synthetic matrices are generated with sizes proportional to the number of MPI processes,

ensuring an approximately equal workload for each process. For all experiments, the input vector is randomly generated and sized to match the number of matrix columns.

TABLE II: Dataset used for the experiments

Graph	Matrix Name	Rows	Columns	Non-zeros	NNZ [%]
	fd12	7 500	7 500	28 462	0.05
	sinc12	7 500	7 500	283 992	0.51
	epb1	14 724	14 724	95 053	0.04
	c-46	14 913	14 913	130 397	0.06
	human_gene2	14 340	14 340	18 068 388	8.79

V. RESULTS AND DISCUSSION

In this section, we evaluate the performance of SpMV under different matrix decomposition strategies, processor counts, and matrix characteristics. All execution time measurements were repeated 100 times, and the 90th percentile was used to mitigate outliers. When present, the point at one MPI process corresponds to sequential execution.

A. (Strong) Scaling

Figures 1(a-b) report the strong scaling behavior for matrices with different sparsity structures. For irregular matrices such as c-46, the 1D cyclic decomposition outperforms both 1D and 2D block partitionings. This is explained by its superior load balancing: when using 128 MPI processes, the minimum and maximum number of nonzeros per rank are 720 and 2.152 for 1D cyclic, while for both block decompositions they range from 234 to 34.748. Since the reported SpMV time is taken as the maximum across all ranks, this imbalance directly limits strong-scaling efficiency, making cyclic decomposition preferable for highly irregular patterns. For more regular and diagonal matrices such as epb-1, both 1D and 2D block decompositions provide better performance thanks to the sparsity pattern that provides an already balanced workload. The cyclic distribution, although balanced in terms of nonzeros, destroys cache locality due to scattered accesses and increased memory traffic.

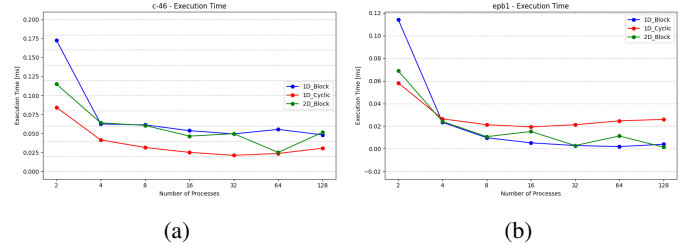


Fig. 1

B. (Weak) Scaling

Figure 2(a) shows the weak scaling behavior as both the matrix size and the number of MPI processes increase proportionally. As already observed previously, block-based decompositions consistently outperform the cyclic one for more regular matrices due to better cache locality. From Figure 2(b), 1D and 2D block decompositions show very similar memory usage; however, as shown in Figure 2(c), 2D block achieves higher FLOPS at large scale, peaking at approximately 18 GFLOPS with 64 MPI processes. This advantage comes from the more favorable structure and locality of the local submatrices produced by the two dimensional partitioning, which improves cache utilization and memory bandwidth efficiency of the SpMV kernel.

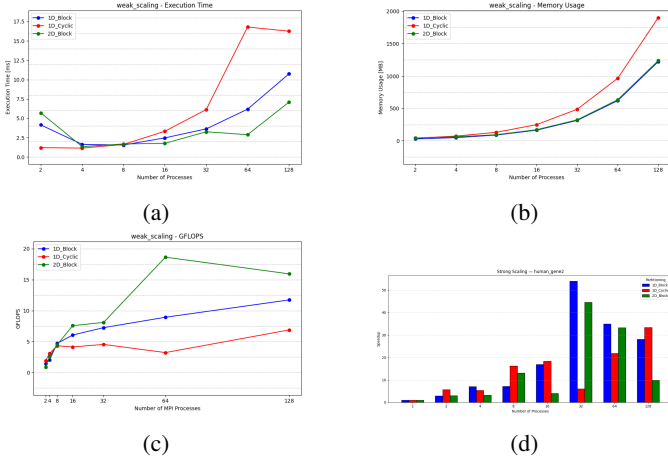


Fig. 2: (a-c) weak scaling results; (d) human_gene2 speedup

C. Performance Analysis

To further analyze performance limits, we consider the larger and denser human_gene2 matrix. Figure 2(d) shows that 1D block partitioning achieves the highest speedup, peaking at more than 50x at 32 MPI processes compared to sequential execution. The corresponding efficiency plot in Figure 3(a) indicates that efficiency exceeds 1 at some points due to superlinear speedup, which can be attributed to improved cache utilization and reduced memory traffic in parallel executions compared to the sequential baseline. The 2D block decomposition shows slightly lower peak speedup but degrades more gradually at larger process counts, indicating better scalability. On the other hand, 1D cyclic partitioning collapses at 32 processes: although it provides good load balance, it significantly degrades cache locality. At 128 processes its performance partially recovers due to a better workload distribution ($\approx 75,000$ difference between minimum and maximum nonzeros per rank versus $\approx 140,000$ at 32 processes). Figure 3(b) shows that communication time flattens at higher process counts, revealing a latency-dominated behavior typical of strong-scaling SpMV. Memory usage trends are reported in Figure 3(c). The 1D block decomposition shows the lowest peak memory footprint, similarly followed by the 2D block

approach. The 1D cyclic decomposition requires more memory due to worse cache locality caused by the round-robin row distribution. Additionally, the memory curves flatten at higher process counts because the root process stores and distributes the entire COO matrix, which dominates the maximum per-process memory footprint. Finally, the GFLOPS results in Figure 3(d) confirm that, for structured matrices, block-based decompositions have significantly better performance. The cyclic strategy, while beneficial for irregular matrices due to load balance, remains limited by poor data locality and higher communication overhead.

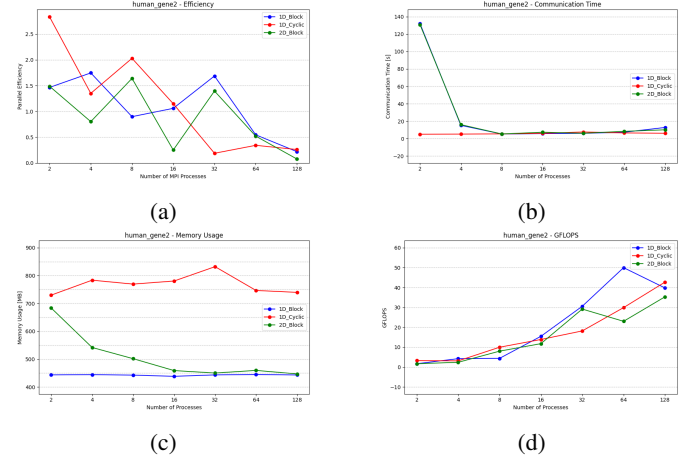


Fig. 3

VI. CONCLUSIONS

In this study, we evaluated the performance of distributed memory Sparse Matrix-Vector Multiplication (SpMV) on a multi-socket x86_64 server using five real matrices for strong scaling experiments and synthetic matrices for weak scaling, considering multiple matrix decomposition strategies implemented with MPI. Our results show that 1D cyclic partitioning, despite poor cache locality, performs better for highly irregular matrices by partially balancing the workload across MPI processes. In contrast, both 1D and 2D block decomposition strategies deliver better performance on regular matrices due to balanced workloads per process and better cache locality. These results are strengthened by memory footprint measurements, which indicate higher memory usage for 1D cyclic decomposition and lower usage for block-based strategies. The experiments also highlight that increasing the number of MPI processes does not necessarily result in a proportional speedup. These results highlight the importance of selecting an appropriate decomposition strategy based on matrix structure and system architecture. Future work could explore hybrid MPI+OpenMP implementations, and investigate other decomposition strategies for highly irregular sparse matrices to further optimize performance and memory usage.

VII. REPRODUCIBILITY & ARTIFACT AVAILABILITY

All code, datasets, and plots used in this study are publicly available at the following GitHub repository: [7].

REFERENCES

- [1] K. Hou, W. -c. Feng and S. Che, "Auto-Tuning Strategies for Parallelizing Sparse Matrix-Vector (SpMV) Multiplication on Multi- and Many-Core Processors," 2017 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), Lake Buena Vista, FL, USA, 2017, pp. 713-722, doi: 10.1109/IPDPSW.2017.155. keywords: Kernel;Sparse matrices;Graphics processing units;Computational modeling;Systems architecture;Programming;SpMV;GPU;APU;AMD;Machine Learning;auto-tuning;performance tuning;CSR;binning;classification;graph processing,
- [2] A. A. K. Al-Saeedi, S. O. Yurievna and T. W. Khairi, "Improving the efficiency of sparse matrix class processing by using the SPM-CSR parallel algorithm and OpenMP technology," 2022 4th International Youth Conference on Radio Electronics, Electrical and Power Engineering (REEPE), Moscow, Russian Federation, 2022, pp. 1-4, doi: 10.1109/REEPE53907.2022.9731504. keywords: Power engineering;Instruction sets;Software algorithms;Systems architecture;Software;Sparse matrices;Parallel algorithms;OpenMP;SpMV;reduction;sparse matrix;CSR,
- [3] N. Kasmi, S. A. Mahmoudi, M. Zbakh and P. Manneback, "Performance evaluation of sparse matrix-vector product (SpMV) computation on GPU architecture," 2014 Second World Conference on Complex Systems (WCCS), Agadir, Morocco, 2014, pp. 23-27, doi: 10.1109/ICoCS.2014.7060964. keywords: Graphics processing units;Equations;Clocks;Instruction sets;Programming;Arrays;Sparse matrix;SpMV;GPU;Cusp;MKL;CUDA;Sparse Matrix Vector Product,
- [4] E. G. Boman, K. D. Devine and S. Rajamanickam, "Scalable matrix computations on large scale-free graphs using 2D graph partitioning," SC '13: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis, Denver, CO, USA, 2013, pp. 1-12, doi: 10.1145/2503210.2503293. keywords: Sparse matrices;Vectors;Partitioning algorithms;Layout;Eigenvalues and eigenfunctions;Symmetric matrices;Software;parallel computing;graph partitioning;scale-free graphs;sparse matrix-vector multiplication;two-dimensional distribution,
- [5] J. Chen, Y. Dong, Y. Li and W. Feng, "Parallel Computing Optimization for Ground-based TT&C Network Situational Data Processing," 2021 2nd International Conference on Computer Science and Management Technology (ICCSMT), Shanghai, China, 2021, pp. 297-304, doi: 10.1109/ICCSMT54525.2021.00063. keywords: Computer science;Graphics processing units;Parallel processing;Data processing;Real-time systems;Task analysis;Aerospace industry;parallel computing;SIMD;OpenMP;MPI;matrix multiplication,
- [6] Kolodziej et al., (2019). The SuiteSparse Matrix Collection Website Interface. Journal of Open Source Software, 4(35), 1244, <https://doi.org/10.21105/joss.01244> Available online at: <https://sparse.tamu.edu/>
- [7] <https://github.com/FrancescoBrunelli/PARCO-Computing-2026-245252.git>